

공부 내용 정리 (1장, 2장)

CHAPTER 1

트랜스포머 소개

구글

2017

시퀀스 모델링을 위한 새로운 신경망 아키텍처 제안

Transformer

RNN(recurrent neural network, 순환 신경망) 능가

품질, 훈련 비용 면에서

ULMFiT: 효율적 전이 학습(transfer learning) 방법

ULMFiT가 매우 크고 다양한 corpus(말뭉치)에서 LSTM 신경망을 훈련해

매우 적은 양의 레이블링된 데이터로도 최고 수준의 텍스트 분류 모델을 만들어냄을 입증

GPT, BERT의 촉매

trnasformer 아키텍처와 unsupervised learning 결합

작업 특화 모델을 밑바닥부터 훈련할 필요 없음



그림 1-1 트랜스포머 모델의 타임라인

transformer의 새로운 점을 이해하기 위해

- encoder-decoder 프레임워크
- attention mechanism
- 전이 학습(transfer learning)

알아야 함

1.1 encoder-decoder framework

hidden state, 은닉상태

RNN은 입력을 받으면 hidden state라는 벡터를 출력

동시에 피드백 루프로 보내 자기 자신에게 입력

RNN의 능력에 대해 Andrej Karpahty가 설명한 블로그 참고

RNN은 단어 시퀀스를 한 언어에서 다른 언어로 매핑하는 기계 번역 시스템에서 중요한 역할

이런 작업은 주로 encoder-decoder, sequence-to-sequence 구조로 처리

입력과 출력이 임의의 길이를 가진 sequence일 때 잘 맞음

입력 시퀀스 정보 – encoder(encoding) → last hidden state, 마지막 은닉 상태, 수치 표현 – decoder → 출력 시퀀스

encoder와 decoder는 sequence를 모델링할 수 있는 어떤 종류의 신경망도 가능

간결하지만, encoder의 last hidden state가 information bottleneck, 정보 병목이 된다는 약점 존재

sequence가 길면 last hidden state에 전체 입력 sequence가 다 담겨야 해서 정보가 손실될 가능성 존재

attention 매커니즘

decoder가 encoder의 모든 hidden state에 접근해서 bottleneck을 제거

1.2 attention mechanism

입력 sequence에서 hidden state를 만들지 않고, 스텝마다 encoder에서 decoder가 참고할 hidden state를 출력

모든 상태를 동시에 사용하려면 decoder에 많은 입력이 발생 -> 우선순위 정해야 함

우선순위 정하는 mechanism: attention

decoder가 모든 decoding timestep마다 encoder의 state에 다른 가중치, attention을 할당

encoder와 decoder에 사용하는 순환 모델은 태생적으로 계산이 순차적으로 수행 -> 입력 sequence 전체에 걸쳐 병렬화할 수 없음

transformer가 모델링 패러다임을 바꿈

순환을 없애고, self-attention(CHAPTER 3에서)으로

self-attention

신경망의 같은 층에 있는 모든 상태에 대해 attention을 작동시키는 방식

encoder, decoder 각각에 self-attention 있음

attention의 출력이 FF NN, feed-forward neural network에 주입

순환 모델보다 빠르게 훈련

최근 NLP 분야의 혁신

레이블된 대규모 corpus가 있어야 학습시킬 수 있음

하지만 없기 때문에 transfer learning 나타남

1.3 NLP의 transfer learning

computer vision에서 ResNet 같은 합성곱 신경망을 한 작업에서 훈련하고 새로운 작업에 적용하거나 fine-tuning하는 경우 많음

신경망은 원래 작업에서 학습한 지식 사용

body의 가중치는 훈련하는 동안 원래 domain에서 다양한 특성을 학습

이 가중치를 활용해 새로운 작업을 위한 모델을 초기화

supervised learning과 비교해서 transfer learning은

일반적으로 다양한 작업에서 적은 양의 레이블 데이터로 훨씬 효과적으로 훈련하는 높은 품질의 모델을 만듦

ImageNet: 이미지가 수백만 개인 데이터셋. computer vision에서 사용

대규모 데이터셋을 학습하는 과정이 pretraining, 사전 훈련

모델에게 엣지나 색깔 같은 기본적인 이미지 특징을 가르치는 것이 그 목적

NLP에서 transfer learning과 유사한 학습 과정이 뭔지 몰랐음

그래서 레이블링된 대량의 데이터를 사용했지만, computer vision보다 성능 낮았음

OpenAI 연구원들이 감성 분류 작업에 unsupervised pretraining에서 추출한 특성을 사용해 높은 성능을 얻으며 소개됨

그 뒤를 이어 범용적인 framework ULMFiT가 등장

ULMFiT는 다양한 작업에서 pretraining된 LSTM 모델 적용

사전 훈련, pretraining

초기 훈련의 목표는 이전 단어로 다음 단어를 예측하는 것

language modeling, 언어 모델링 이라고 함

레이블링된 데이터 불필요

위키피디아 같은 소스에 있는 풍부한 텍스트 활용

도메인 적응

language model을 대규모 corpus에서 pretraining하고, domain 내 corpus에 적응시킴

여전히 language modeling을 사용하지만, IMDb(특정 domain)에 있는 다음 단어를 예측
fine-tuning, 미세 튜닝

NLP에서 pretraining과 transfer learning을 수행하기 위해 ULMFiT이 제시한 프레임워크가 transformer가 부상하기 위한 마지막 조각이었음

2018

self-attention + transfer learning

결합한 두 모델 릴리스

GPT: transformer 아키텍처의 decoder 부분만 사용

BERT: masked language model

1.4 Hugging Face Transformers

머신러닝 아키텍처를 새로운 작업에 적용하는 방법

- 모델 아키텍처를 코드로 구현 (주로 PyTorch, TensorFlow 사용)
- 가능하다면 서버에서 사전 훈련된 가중치 로드
- 입력을 전처리하고 모델에 전달. 이후 해당 작업에 맞는 사후 처리 수행
- dataloader, 데이터로더를 구현하고 모델 훈련을 위해 loss function, 손실함수와 optimizer, 옵티마이저를 정의

1.5 transformer 애플리케이션 둘러보기

NLP에서 제품, 장소, 사람 같은 실제 객체를 개체명, named entity라고 함

이런 named entity를 텍스트에서 추출하는 과정을 named entity recognition, NER이라 함

extractive question answering, 추출적 질문 답변: 답변을 텍스트에서 직접 추출

1.7 transformer의 주요 도전 과제

다양한 언어(4장)
레이블링 데이터 없거나 소량인 경우(9장)
문서 수준의 긴 텍스트에서 비용 과다(11장)
불투명성. 모델의 오류를 조사하는 방법(2장, 4장)
인터넷 학습으로 인한 편향(10장)

CHAPTER 2

텍스트 분류

DistilBERT를 활용한 감성 분석

BERT보다 훨씬 작고 효율적

훈련이 몇 분 만에 됨

대규모 BERT 모델을 훈련해야 한다면, 간단히 사전 훈련된 모델의 checkpoint를 바꾸면 됨

2.1 데이터셋

특정 논문 데이터셋 사용

트윗의 글을 감정 6가지로 나눈 데이터셋

list_datasets() 함수를 사용하면 허브에서 제공하는 데이터셋 목록을 리스트로 만들 수 있음

데이터셋을 데이터프레임으로

클래스 분포 살펴보기

소수 클래스를 랜덤으로 오버샘플링

다수 클래스를 랜덤으로 언더샘플링

클래스의 대표성이 부족하면 레이블된 데이터를 더 많이 수집

일반적으로 훈련 세트에만 샘플링을 적용함

훈련/테스트 분할을 만들기 전에는 샘플링 하지 않기

2.2 텍스트에서 토큰으로

신경망은 가상의 순서 같은 종류의 관계를 잘 학습함

따라서 one-hot vector 방식을 사용

tensor: 숫자 덩어리, 배열

문자 토큰화

철자 오류, 희귀한 단어 학습에 유리

단어 같은 언어 구조를 이 데이터 안에서 학습해야 함

계산, 메모리, 데이터 많이 소모

2.3 텍스트 분류 모델 훈련하기

transformer로 특성 추출

transformer fine-tuning